ARCHITECTURE & VERIFICATION STANDARD

독립 서브에이전트 기반 적대적 코드 심사 및 5대 최적화 검증 체계 구현 보고서

Lauren Tan(pstack) 멀티모델 검증 패러다임의 UE5.8 데디케이트 서버 복제 아키텍처 특화 구현

프로젝트	환경	구현 모듈	일자
FC (Unreal Engine 5.8)	Google Antigravity	/interrogate & adversarial-auditor	2026. 10. 28

Executive Summary
본 문서는 Lauren Tan(SpaceX/Cursor)의 `pstack` 멀티모델 교차 검증 패러다임(`/interrogate`, `/arena`)을 분석하고, 이를 FC 프로젝트의 Unreal Engine 5.8 데디케이트 서버 아키텍처에 맞추어 구현한 **적대적 코드 심사관(Adversarial Code Auditor)** 시스템을 기술합니다. 단일 생성 모델의 인지적 맹점(Blind Spot), 오버엔지니어링, 복제 누락, 불필요한 틱 활성화를 원천 차단하기 위해, 읽기 전용(Read-Only) 서브에이전트가 **5대 핵심 차원(간결성, 기존 구조 활용, 범위 내 적정 확장성, 네트워크 최적화, 일반 최적화)**을 기준으로 코드를 엄격히 심사합니다.

1. 배경 및 도입 목적

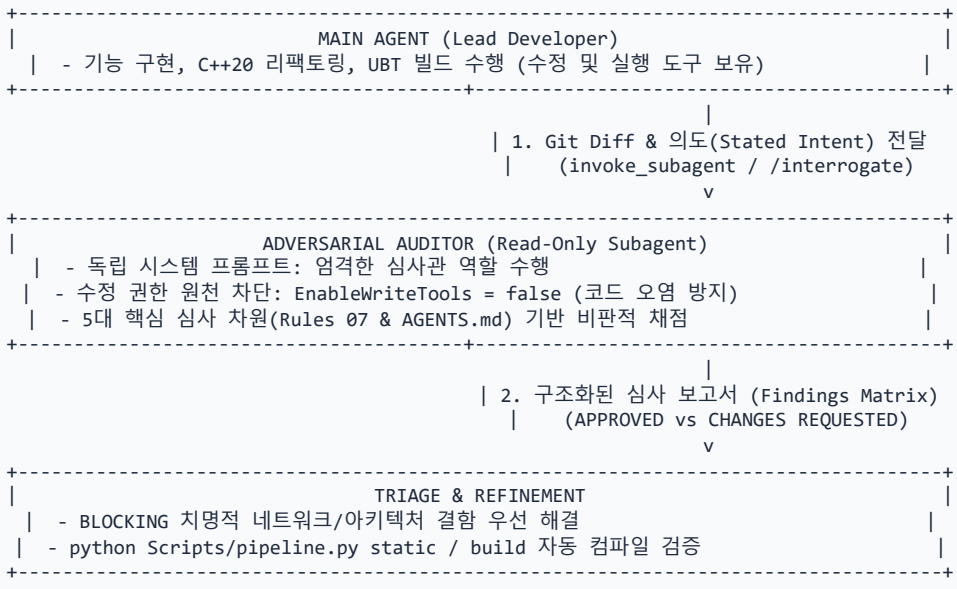
1.1 단일 생성 모델(Single Agent)의 확증 편향 극복

단일 AI 모델이 코드를 작성하는 경우, 본인이 작성한 컨텍스트 내에서 확증 편향이 발생하여 소유권 없는 액터에서의 Server RPC 선언, 불필요한 레이어 래핑, GC 추적 누락과 같은 미묘한 버그를 스스로 인지하지 못하는 한계가 있습니다.

1.2 언리얼 엔진 5.8 멀티플레이어 환경의 특수성

C++ 컴파일러나 일반 린터는 문법적 정합성만 검사할 뿐, **서버 권한 분리(HasAuthority), RPC Connection 소유권, 복제 조건(DOREPLIFETIME_CONDITION), 가비지 컬렉션(TObjectPtr) 수명 주기**와 같은 도메인 특화 결함을 사전에 탐지하지 못합니다. 따라서 커밋 전 엄격한 루브릭을 갖춘 독립 심사 서브에이전트의 교차 검증이 필수적입니다.

2. 독립 서브에이전트 아키텍처 다이어그램



3. 5대 핵심 심사 차원 (The 5 Core Audit Dimensions)

심사 차원	핵심 점검 기준	차단되는 안티패턴
1. 코드 간결성 (Conciseness)	최소 표면 수정(Minimal Surface Area), 명료한 C++20 관용구 사용, 불필요한 보일러플레이트 배제	1회성 로직을 위한 무분별한 래퍼 함수, 과도한 인터페이스 다형성 남발
2. 기존 구조 활용 (Maximal Reuse)	기존 Subsystem, CombatStatics, DataTable, Component 재사용, SSOT 데이터 자산 준수	기존 파이프라인 무시 후 병렬 클래스 생성, loose *.csv 파일 런타임 직접 I/O
3. 범위 내 확장성 (Scoped Extensibility)	**주어진 기획 스펙 범위**에 최적화된 유연성, YAGNI 원칙 엄수, 데이터 기반 잎새 확장 보장	기획에 없는 먼 미래를 가정한 과설계(Over-engineering), 과도한 추상 팩토리 계층
4. 네트워크 최적화 (Network & Replication)	HasAuthority() 상태 변조 가드, Connection 소유 액터의 Server RPC, DOREPLIFETIME_CONDITION, RepNotify 우선	소유권 없는 월드 액터의 Server RPC, 틱 단위 고빈도 RPC 전송, 조건 없는 무분별한 복제
5. 일반 성능 최적화 (General Optimization)	PrimaryActorTick.bCanEverTick = false 기본화, 핫패스 힙 할당 제로, TObjectPtr 및 TWeakObjectPtr GC 위생	Tick() 내 TArray::Add 또는 문자열 포맷팅, UObject 스마트 포인터 순환 참조 릭

4. 워크플로우 및 프로토콜 규격 (/interrogate)

> **실무 호출 가이드**: 대화창에 `/interrogate` 또는 "코드 검사해줘", "최적화 검사해줘" 입력 시, 메인 에이전트가 `git diff`를 취합하여 `adversarial-auditor` 서브에이전트를 호출합니다.

4.1 심사관 서브에이전트 반환 리포트 규격 예시

```
# 🛡️ Adversarial Audit Report
Verdict: [CHANGES REQUESTED (BLOCKING)]

### Findings Matrix
| Severity | Dimension | Location | Issue & Root Cause | Actionable Fix |
| :--- | :--- | :--- | :--- | :--- |
| BLOCKING | Network | Source/FC/...:42 | Server RPC on unowned World Actor | APlayerController로 RPC 이전 |
| WARNING | Reuse | Source/FC/...:88 | Re-implemented math helper | FCCombatStatics 유틸리티 재사용 |
| SUGGESTION | Conciseness | Source/FC/...:120 | Redundant temporary wrapper | 인라인화 처리로 간결화 |

### Dimension Scoring
- Conciseness: [9/10]  |  Architecture Reuse: [7/10]  |  Scoped Extensibility: [8/10]
- Network Optimization: [6/10]  |  General Optimization: [9/10]
```

5. 기대 효과 및 결론

- **무결점 멀티플레이어 보장**: 런타임에서 패킷 드롭이나 Desync를 유발하는 RPC 소유권 위반과 권한 체크 누락을 빌드 전 사전 박멸.
- **아키텍처 부채 제로화**: 주어진 기획 범위를 초과하는 과설계를 방지하고 기존 서브시스템을 최우선 재사용하여 코드 비대화 차단.
- **안전한 자율 에이전트 루프 완성**: 작업 에이전트와 심사 에이전트 간의 견제와 균형을 통해 상용 수준의 C++20 엔지니어링 퀄리티 달성.